PROJECT REVIEW III – FINAL REVIEW

Progress, Individual Contribution and Project Outcome Report

KELVIN: AN AI-ASSISTED DOMAIN-SPECIFIC LANGUAGE COMPILER FOR IoT AUTOMATION

BCSE307P – Compiler Design Lab

Assessment 10

Submitted by

Name	Register Number
R. Sanjay	24BCI0181
Settipalli Tejovardhan	24BCI0196
Ashutosh	23BCI0117

Under the guidance of

Prof. Sathya K

VELLORE INSTITUTE OF TECHNOLOGY

Vellore – 632014, Tamil Nadu, India

October 2026

SUMMARY

This document accompanies our final review. The project, Kelvin, is complete: every compiler phase, the AI assistant with its verification gate, the four code generators, the web studio and the digital twin all work, and the full test suite of 145 tests passes. Section 1 records the completion status module by module. Section 2 describes what each of the three of us built and the topics each person can explain during the viva. Section 3 covers the project outcome required for this review: the target conference, the manuscript prepared in the IEEE conference template (included in this document as Annexure A), the measurable results from a new experiment we ran for the paper, and how our work compares with recent research. Section 4 explains how we kept the written material original.

TABLE OF CONTENTS

SUMMARY i
1. PROJECT COMPLETION STATUS 1
2. INDIVIDUAL CONTRIBUTION 2
2.1 R. Sanjay (24BCI0181) 2
2.2 Settipalli Tejovardhan (24BCI0196) 2
2.3 Ashutosh (23BCI0117) 3
3. PROJECT OUTCOME 4
3.1 Target Venue 4
3.2 Manuscript 4
3.3 Measurable Outcomes 4
3.4 Novelty Compared with Recent Research 5
4. INTEGRITY OF THE SUBMITTED WORK 6
ANNEXURE A: MANUSCRIPT 7

1. PROJECT COMPLETION STATUS

All planned work has been implemented and tested. Table 1 lists each module with the evidence that it works. Everything can be demonstrated live from one laptop with `make run`, and offline if no AI service is reachable.

Table 1: Completion status of every module

Module	Status	Evidence
Lexical analysis (DFA scanner)	Complete	Token table, DFA drawn in the studio, lexical error tests
Syntax analysis (LL(1) parser)	Complete	Grammar verified by the compiler: 61 productions, 0 conflicts
Abstract syntax tree	Complete	Tree view and graph in the studio
Semantic and rule analysis	Complete	Type errors, suggestions, conflict warnings with example inputs
Intermediate code (TAC)	Complete	Original and optimized listings
CFG, dataflow and optimizer	Complete	8 passes; example 07 shrinks from 21 to 10 instructions
Code generation (4 targets)	Complete	C, Arduino, Python and JavaScript agree in tests
AI assistant and verification gate	Complete	96.7% correct on our 30-rule benchmark
Web studio, X-ray, digital twin	Complete	Live demo; phone remote through QR code
Testing	Complete	145 automated tests, all passing
Manuscript (IEEE template)	Complete	Annexure A of this document

2. INDIVIDUAL CONTRIBUTION

We divided the work by compiler module so that each of us owned a part of the pipeline from design to testing. We reviewed each other's code and all three of us can run the full demo, but the areas below are where each person did most of the work and can answer detailed questions.

Table 2: Division of work

Member	Register No.	Main contribution
R. Sanjay	24BCI0181	Lexer, parser, syntax tree, grammar analysis, web studio
Settipalli Tejovardhan	24BCI0196	Semantic and rule analysis, intermediate code, CFG optimizer, code generation
Ashutosh	23BCI0117	AI assistant and verification gate, test suite, benchmark, documentation

2.1 R. Sanjay (24BCI0181)

Sanjay built the front of the compiler. He wrote the scanner as a hand-coded finite automaton with longest-match tokenisation and position tracking, and added the lexical error messages with their quick fixes. He then wrote the recursive-descent parser, designed the grammar so that it is LL(1), and implemented panic-mode recovery so that several syntax errors can be reported at once. He also wrote the grammar-analysis module that computes FIRST and FOLLOW sets and the LL(1) table from the grammar stored inside the compiler. On the interface side he built the studio's editor, the syntax-tree graph and the Compiler X-ray that replays the scanner and parser traces.

Can explain: DFA construction and maximal munch; FIRST/FOLLOW computation and why the grammar has no LL(1) conflicts; recursive descent and operator precedence; error recovery; how parser traces drive the X-ray.

2.2 Settipalli Tejovardhan (24BCI0196)

Tejovardhan built the middle and back end. He implemented the symbol table and type checking, the *did-you-mean* suggestions, and the rule analysis that converts conditions to disjunctive normal form and uses intervals and value sets to find impossible, always-true, unreachable and conflicting rules. He designed the three-address intermediate code, split it into basic blocks, built the control-flow graph, and wrote the liveness and available-expressions analyses used by the eight optimization passes. He also wrote the structured emitter and the C, Arduino, Python and JavaScript back ends with their runtimes.

Can explain: type rules; DNF and the satisfiability test used for conflicts; three-address code; basic blocks and CFG construction; dataflow equations and fixed-point iteration; each optimization pass; recovering if–else structure during code generation.

2.3 Ashutosh (23BCI0117)

Ashutosh built the AI side and the verification around it. He integrated the three free model providers (Ollama, Groq and Gemini), wrote the prompts, and implemented the verification gate: compiler-guided repair, the meaning check that catches dropped numbers and unused sensors, and the fallback to the rule-based converter, which he also wrote. He set up the test suite and the cross-target equivalence tests, designed and ran the 30-rule benchmark used in the manuscript, measured the compiler's timing, and prepared the reports and the manuscript.

Can explain: how the gate decides to accept, repair or reject an answer; why a meaning check was needed; how behavioural correctness is measured in the benchmark; the test strategy; the experimental results and their limitations.

3. PROJECT OUTCOME

3.1 Target Venue

We prepared the manuscript for an IEEE conference using the official two-column IEEE conference template. Our first choice is the **IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)**, which accepts work on IoT systems and applied software and publishes accepted papers through IEEE Xplore. Our second choice is **IEEE INDICON**, the annual conference of the IEEE India Council. Both are held every year in India and accept papers in this format. Before submitting, we will check the current call for papers for the exact deadline, page limit and author guidelines, and adjust the manuscript if needed.

3.2 Manuscript

Table 3: Manuscript details

Item	Details
Title	Kelvin: Compiler-Verified Natural-Language Programming of IoT Automation Rules
Authors	R. Sanjay, Settipalli Tejovardhan, Ashutosh, Sathya K
Format	IEEE conference template, two columns, US Letter
Length	5 pages, 4 figures, 3 tables, 20 references
Sections	Introduction, Related Work, Rule Language, Compiler Design, Verification Gate, Implementation, Evaluation, Discussion and Novelty, Conclusion
Location	Annexure A of this document

3.3 Measurable Outcomes

For the paper we ran a new experiment. We wrote 30 English automation rules and, for each, a reference program. Every answer was compiled to Python and run on up to 60 test inputs; it counted as correct only if it switched every device exactly as the reference did. We compared the rule-based converter, the raw output of the local Qwen3 8B model, the model with compiler-error repair, and the complete Kelvin gate.

Table 4: Results of the English-to-DSL experiment

Pipeline	Compiles	Behaviourally correct
Rule-based converter only	83.3%	83.3%
LLM output, unchecked	96.7%	90.0%
LLM with compiler-error repair	100.0%	93.3%
Kelvin verification gate	**100.0%**	**96.7%**

Other measured results reported in the manuscript:

- The gate raised accuracy by 6.7 percentage points over unchecked model output while adding only about 0.1 s to the median conversion time.
- The compiler finishes all seven phases in under 0.6 ms on each of the 13 example programs.
- The optimizer removes 52% of the instructions in the optimization example.
- All 145 regression tests pass, including tests that the C, Python and JavaScript outputs behave identically.

3.4 Novelty Compared with Recent Research

Table I of the manuscript compares Kelvin with eight systems published between 2018 and 2024. Safety analysers such as Soteria (2018), AutoTap (2019) and HAWatcher (2021) study existing smart-home programs but do not take English input. Recent work on checking model output, such as LEVER (2023) and Self-Debugging (2024), relies on executing the generated code, while Synchromesh (2022) and grammar-constrained decoding (2023) only guarantee correct syntax. Sasha (2024) applies language models to smart homes without checking the result against a language definition.

Kelvin differs in three ways. First, a complete domain compiler, not just a syntax check or a test run, decides whether a model's answer is accepted. Second, the compiler finds conflicting and unreachable rules at compile time and gives an example input for each. Third, one verified program is compiled to code for microcontrollers as well as for Python and JavaScript. The experiment shows that this design produces measurably more correct rules than using the model alone.

4. INTEGRITY OF THE SUBMITTED WORK

The manuscript and this report were written by our team and describe our own system. The results come from experiments we ran on our own code, and the scripts used to produce them are kept in the project so that they can be repeated. Every outside idea is cited, and the manuscript was written separately from our Review II report rather than copied from it, so that the two documents do not overlap.

The review guidelines require a Turnitin similarity of at most 10% and AI-generated content of at most 20%. We will run both documents through Turnitin before the final submission and revise any passage that is flagged.

ANNEXURE A: MANUSCRIPT

The following pages contain the manuscript prepared for submission, in the IEEE two-column conference format. Its figures, tables and references are numbered independently of this report.

Kelvin: Compiler-Verified Natural-Language Programming of IoT Automation Rules

R. Sanjay
Dept. of Computer Science and Engg.
Vellore Institute of Technology
Vellore, India

Settipalli Tejovardhan
Dept. of Computer Science and Engg.
Vellore Institute of Technology
Vellore, India

Ashutosh
Dept. of Computer Science and Engg.
Vellore Institute of Technology
Vellore, India

Sathya K
Dept. of Computer Science and Engg.
Vellore Institute of Technology
Vellore, India

***Abstract*—**Trigger–action rules are the usual way to automate homes and small buildings, and large language models (LLMs) now let people write such rules in plain English. The catch is that a model's answer can be syntactically valid and still not do what the user asked. We present Kelvin, a system in which an LLM only proposes a rule and a hand-written compiler for a small IoT rule language decides whether it is accepted. The compiler covers the full pipeline: a finite-automaton scanner, an LL(1) recursive-descent parser whose grammar is checked for conflicts by the compiler itself, type checking, a satisfiability-based analysis that reports conflicting and unreachable rules, three-address code optimized over a control-flow graph, and back ends for C, Arduino, Python and JavaScript. Between the model and the compiler sits a verification gate that combines compiler diagnostics, a lightweight meaning check and one guided repair. On a benchmark of 30 English rules with a local 8B-parameter model, behavioural accuracy rose from 90.0% for unchecked model output to 96.7% with the gate, every accepted program compiled, and the median conversion time stayed near 2.1 s. The compiler handles each of 13 example programs in under 0.6 ms.

***Index Terms*—domain-specific languages, compilers, trigger-action programming, Internet of Things, large language models, program verification.**

I. Introduction

A large share of home automation is expressed as short trigger–action rules: when a sensor crosses a threshold, switch a device. Field studies have shown that people without programming experience can write such rules [1], yet they regularly misjudge what their rules will do once several of them interact [2]. A natural next step is to let users describe rules in English and have an LLM produce the program. Code models are capable but not dependable; a sizeable fraction of their programs fail even simple tests [3], and in a house a wrong rule has physical consequences, such as a heater left off or a door left unlocked [4], [7].

Our position is that the model should never be the final authority. In Kelvin the LLM writes a candidate rule in a small domain-specific language (DSL), and a conventional compiler, written by us from scratch, decides whether the candidate is acceptable. This lets well-understood compiler techniques do the checking: lexical and syntactic analysis, type rules, static reasoning about conditions, and dataflow-based optimization.

The contributions of this paper are:

1) a compact rule DSL with an LL(1) grammar that the compiler verifies itself by computing FIRST and FOLLOW sets and the parse table;
2) a compile-time rule analysis that converts conditions to disjunctive normal form and reports impossible, always-true, unreachable and conflicting rules together with a witness input;
3) a verification gate around the LLM that combines compiler errors, a meaning check and a single guided repair, with a rule-based fallback;
4) an evaluation on 30 English rules showing a 6.7-point gain in behavioural accuracy over unchecked model output at almost no extra latency, together with compiler timing and a 145-test regression suite.

II. Related Work

A. Safety of Trigger–Action Programs

Ur et al. [1] established trigger–action programming as a practical model for smart homes, and Brackenbury et al. [2] classified the bugs users make, many of which involve rules that fight over a device. Several tools analyse such programs after they are written. Soteria [4] extracts a state model from smart-home apps and model-checks safety properties, AutoTap [5] synthesizes and repairs rules so that they satisfy temporal-logic properties, and HAWatcher [6] detects anomalies at run time using the semantics of apps and devices. Ding and Hu [7] showed that devices can also interact through the physical environment. Kelvin moves part of this reasoning into the compiler of a rule language, so that conflicts are reported while the rule is being written.

B. Checking the Output of Code Models

Code models such as Codex [3] and Code Llama [8] generate programs from text but give no guarantee of correctness. Self-Debugging [9] feeds execution results back to the model, LEVER [10] trains a verifier that ranks candidates using execution, and Synchromesh [11] and grammar-constrained decoding [12] restrict generation to syntactically valid output. Earlier, Desai et al. [14] synthesized DSL programs from natural language with a ranking model. Sasha [13] applies LLMs to under-specified smart-home commands but relies on the model's own reasoning. Our gate differs in that its feedback comes from a full domain compiler, including type and rule analysis, rather than from execution alone, and it also checks that the answer keeps the numbers and sensors of the sentence.

C. Compiler Foundations

We follow Mernik et al. [15] in building an external DSL with its own parser. Instead of a parser generator such as ANTLR [16] we wrote the parser by hand and had the compiler verify the grammar. The optimizer uses basic blocks and control-flow analysis [17] with Kildall's iterative dataflow framework [18], and, in the spirit of LLVM [19], a single intermediate representation serves every back end.

TABLE I
COMPARISON WITH RECENT RELATED SYSTEMS

System	English input	Domain compiler	Safety / conflict analysis	Output checked before use	Generates device code
Soteria [4] (2018)	No	No	Yes (model checking)	–	No
AutoTap [5] (2019)	No	No	Yes (LTL)	Yes	No
HAWatcher [6] (2021)	No	No	At run time	–	No
Synchromesh [11] (2022)	Yes	Grammar only	No	Syntax only	No
LEVER [10] (2023)	Yes	No	No	Yes (execution)	No
GCD [12] (2023)	Yes	Grammar only	No	Syntax only	No
Self-Debugging [9] (2024)	Yes	No	No	Yes (execution)	No
Sasha [13] (2024)	Yes	No	No	No	No
Kelvin (this work)	**Yes**	**Yes, LL(1)**	**Yes, compile time**	**Yes, compiler + meaning**	**C, Arduino, Py, JS**

III. THE RULE LANGUAGE

A program declares numeric sensors (`NUMBER`), state sensors (`STATE`, with values ON, OFF, OPEN, CLOSED, TRUE, FALSE) and devices, followed by rules. A rule has the form `IF` condition `THEN` actions, with any number of `ELSE IF` branches and an optional `ELSE`. Conditions combine comparisons with `AND`, `OR`, `NOT`, brackets and arithmetic, and the built-in actions are `turn_on`, `turn_off`, `toggle` and `set_level`. The core of the grammar is shown below. In BNF it has 61 productions over 29 nonterminals and 33 terminals, and the compiler's own LL(1) analysis finds 0 conflicts.

```
rule     → IF expr THEN actions
           { ELSE IF expr THEN actions } [ ELSE actions ]
expr     → and_expr { OR and_expr }
and_expr → not_expr { AND not_expr }
not_expr → NOT not_expr | rel_expr
rel_expr → arith [ relop arith ]
arith    → term { ( + | - ) term }
term     → unary { ( * | / ) unary }
action   → IDENT ( [ expr { , expr } ] )
```

Conditions and arithmetic share one expression grammar. Besides keeping the parser small, this removes an LL(1) conflict that appears when a bracketed condition and a bracketed calculation are given separate nonterminals.

IV. COMPILER DESIGN

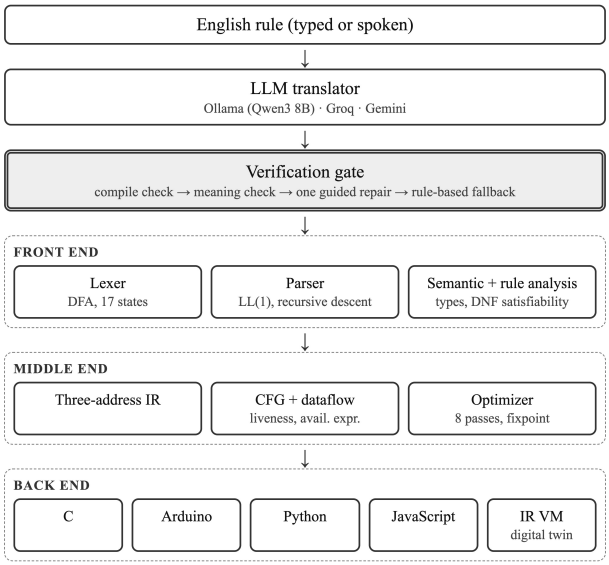


Fig. 1. Architecture of Kelvin. The gate accepts an LLM answer only after it passes the compiler and the meaning check.

A. Front End

The scanner is a hand-coded automaton with 17 states that applies the longest-match rule and records line and column for every token. The parser is recursive descent with one function per nonterminal; after a syntax error it skips to the next `IF`, `SENSOR` or `DEVICE`, so several errors are reported in one pass. Semantic analysis builds a symbol table, assigns each expression the type NUMBER, STATE or BOOL, and checks action arguments. Unknown names receive suggestions based on edit distance, and many diagnostics carry an automatic fix.

B. Rule Analysis

Each condition is rewritten into disjunctive normal form. Within one conjunction, comparisons between a numeric sensor and a constant narrow an interval, and comparisons on a state sensor narrow a set of allowed values. A conjunction is satisfiable when every interval and set is non-empty. With this test the compiler reports (i) conditions that can never hold, (ii) conditions that always hold, (iii) `ELSE IF` branches whose conditions imply an earlier branch, and (iv) pairs of rules that drive the same device to different states when both conditions can be true together. For case (iv) the satisfying assignment is printed, for example *temperature = 31 and door = OPEN*.

Comparisons involving two sensors are treated as unknown, which keeps the analysis sound but incomplete.

C. Intermediate Code and Optimization

Rules are lowered to three-address code with temporaries, labels and conditional jumps, and every instruction keeps its source span. The code is divided into basic blocks and a control-flow graph [17]. Liveness and available expressions are computed iteratively [18]. Eight passes are then applied until no pass changes the code: constant folding, algebraic simplification, common sub-expression elimination, copy propagation, removal of constant branches, removal of unreachable blocks, jump optimization and dead-code elimination.

D. Code Generation

A structured emitter recovers if–else blocks from the optimized code and drives four back ends behind one interface. The C back end links with a small runtime; the Arduino back end produces a sketch with a pin table; the Python and JavaScript back ends ship runtimes that print exactly the same output as the C version, which allows the targets to be tested against each other.

V. Verification Gate

The gate wraps the model as follows. Let s be the English sentence and M the model.

1) Ask M for a DSL program p using a prompt that contains the language reference and the names already declared.
2) Compile p. Collect the compiler errors E. If E is empty, compute meaning issues I: every number in s must occur as a literal in p, and every declared sensor must be used by a rule.
3) If E or I is non-empty, send p back to M once together with the messages and ask for a corrected program p'; recompile.
4) If p' still has errors or meaning issues, try the rule-based converter; accept its output only if it passes both checks. Otherwise return p' marked as unverified.

The meaning check was added after we observed the local model translate *"if the temperature is above 30 and the door is open"* as `IF temperature > 3`. That program compiles, so a gate based only on compiler errors accepts it. The check is deliberately cheap: it needs only the token list and symbol table that the compiler already produces.

VI. Implementation

The compiler consists of 4,865 lines of C++17 in 28 files and uses no external libraries or generators. A server written with the Python standard library exposes it over HTTP, and the model is reached through Ollama (Qwen3 8B [20]), Groq or Gemini. A browser studio (Fig. 2) shows every phase, replays traces recorded by the compiler, and runs the optimized code in a small virtual machine that drives an animated house plan.

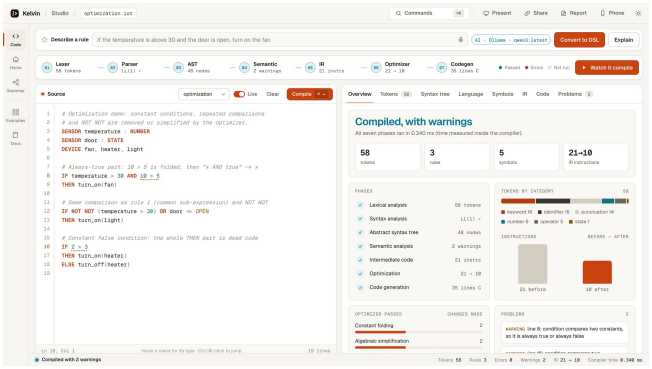


Fig. 2. The Kelvin studio after compiling an example with two warnings; the phase strip and overview are produced from the compiler's JSON output.

VII. Evaluation

A. Setup

All experiments ran on an Apple M3 laptop with 16 GB of memory; the compiler was built with -O2 and the model was Qwen3 8B served locally by Ollama at temperature 0. We wrote 30 English rules covering thresholds, inclusive bounds, conjunctions, disjunctions, negation, levels, several actions per rule and ELSE branches, together with a reference DSL program for each. A candidate counts as correct only if, after compilation to Python, it sets every reference device to the same state as the reference on all test inputs. Inputs are built from the constants of the reference (c−1, c, c+1) and both values of each state sensor, giving up to 60 cases per rule.

B. Natural-Language Conversion

We compare four pipelines: (A) the rule-based converter alone, (B) the raw model answer, (C) the model with compiler-error repair only, and (D) the full gate. Table II and Fig. 3 give the results.

TABLE II
English-to-DSL results on 30 rules

Pipeline	Compiles	Correct	Median time
A Rule-based only	83.3%	83.3%	0.08 ms
B LLM, unchecked	96.7%	90.0%	1.96 s
C LLM + compile check	100.0%	93.3%	2.03 s
D Kelvin gate	**100.0%**	**96.7%**	**2.05 s**

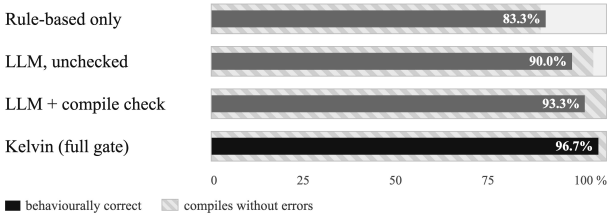


Fig. 3. Behavioural accuracy and compile rate of the four pipelines.

The unchecked model produced one program that did not compile and two that compiled but behaved wrongly. Compile-error repair fixed the first: the model had declared `light` both as a sensor and as a device, and after seeing the compiler's message it removed the duplicate. The meaning check fixed one of the two wrong answers (the dropped condition described in Section V). In all, the gate was triggered on 2 of 30 rules and raised the median time by only 95 ms, because most answers pass on the first attempt. The remaining

failure translated *"it is not raining"* as `rain != ON` while the reference uses TRUE/FALSE. The program is well-typed and uses every number and sensor, so neither check can detect it; it illustrates the limit of purely syntactic meaning checks. The rule-based converter was fastest but failed on phrasings it does not know, such as *"set the fan to level 50"*.

C. Compiler Performance

Table III lists the compiler's own timing for all seven phases (median of 30 runs) on the 13 example programs, including three that contain deliberate lexical, syntax and semantic errors. Time grows roughly linearly with the number of tokens and stays below 0.6 ms. The optimizer shrinks the dedicated optimization example from 21 to 10 instructions (−52%) and removes one to three redundant comparisons in the realistic programs.

TABLE III
COMPILER TIMING ON THE EXAMPLE PROGRAMS

Program	Tokens	IR (before→after)	Time (ms)
basic	23	6→6	0.118
smart home	87	27→26	0.317
greenhouse	52	15→15	0.168
lexical error	12	errors	0.023
syntax error	29	errors	0.045
semantic error	45	errors	0.032
optimization	58	21→10	0.225
rule analysis	86	33→32	0.388
thermostat	96	33→33	0.230
home security	102	34→31	0.325
air quality	118	42→40	0.365
kitchen safety	98	34→32	0.342
office energy	139	52→51	0.567

D. Correctness of the Compiler

A regression suite of 126 compiler checks and 19 unit tests passes in full. It covers each phase, the rule analysis, the quick fixes and the grammar analysis. For every valid example and several inputs it also confirms that the C, Python and JavaScript programs produce identical output and that every Arduino sketch compiles. Fig. 4 shows rule-analysis warnings, with witness inputs, for a program that contains each kind of rule defect.

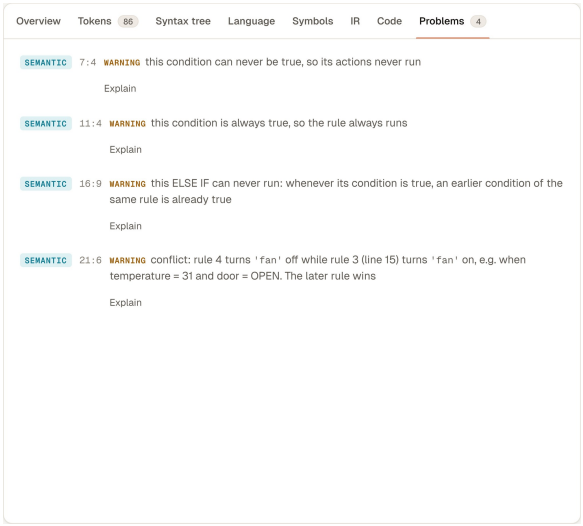


Fig. 4. Rule-analysis warnings, each with an example input, reported at compile time.

VIII. Discussion and Novelty

Table I places Kelvin among recent work. Safety analysers such as Soteria, AutoTap and HAWatcher reason about existing smart-home programs but do not take English input. LLM-based work such as Sasha accepts English but does not check the result against a language definition. Self-Debugging and LEVER verify generated code by running it, and Synchromesh and grammar-constrained decoding guarantee syntax only. As far as we know, Kelvin is the first system to put a complete domain compiler, including compile-time conflict analysis, between an LLM and device code. Its measurable outcomes are a 6.7-point accuracy gain over unchecked output, a 100% compile rate for accepted rules, conflict warnings with witness inputs, and sub-millisecond compilation to four targets.

The study has clear limits. The benchmark is small and written by the authors, a single local model was tested, and the meaning check catches dropped numbers and unused sensors but not wrong choices of state values. The language also lacks timers and persistent state, which many real rules need.

IX. Conclusion

Kelvin shows that a small, carefully specified language and an ordinary compiler can make LLM-written automation rules considerably safer to use. The compiler both verifies the model's output and adds analyses that matter for IoT, such as detecting conflicting rules, before any code reaches a device. Next we plan to add timers and state, to extend the rule analysis to relations between sensors with an SMT solver, to evaluate several models on a larger, independently written benchmark, and to deploy the Arduino output on real boards.

References

[1] B. Ur, E. McManus, M. Pak Yong Ho, and M. L. Littman, "Practical trigger-action programming in the smart home," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, 2014, pp. 803–812.

[2] W. Brackenbury, A. Deora, J. Ritchey, J. Vallee, W. He, G. Wang, M. L. Littman, and B. Ur, "How users interpret bugs in trigger-action programming," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2019, pp. 1–12.

[3] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[4] Z. B. Celik, P. McDaniel, and G. Tan, "Soteria: Automated IoT safety and security analysis," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2018, pp. 147–158.

[5] L. Zhang, W. He, J. Martinez, N. Brackenbury, S. Lu, and B. Ur, "AutoTap: Synthesizing and repairing trigger-action programs using LTL properties," in *Proc. 41st Int. Conf. Software Engineering (ICSE)*, 2019, pp. 281–291.

[6] C. Fu, Q. Zeng, and X. Du, "HAWatcher: Semantics-aware anomaly detection for appified smart homes," in *Proc. 30th USENIX Security Symp.*, 2021.

[7] W. Ding and H. Hu, "On the safety of IoT device physical interaction control," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 832–846.

[8] B. Rozière *et al.*, "Code Llama: Open foundation models for code," arXiv:2308.12950, 2023.

[9] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[10] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. I. Wang, and X. V. Lin, "LEVER: Learning to verify language-to-code generation with execution," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.

[11] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, "Synchromesh: Reliable code generation from pre-trained language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[12] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[13] E. King, H. Yu, S. Lee, and C. Julien, "Sasha: Creative goal-oriented reasoning in smart homes with large language models," *Proc. ACM Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 8, no. 1, 2024.

[14] A. Desai, S. Gulwani, V. Hingorani, N. Jain, A. Karkare, M. Marron, S. R, and S. Roy, "Program synthesis using natural language," in *Proc. 38th Int. Conf. Software Engineering (ICSE)*, 2016, pp. 345–356.

[15] M. Mernik, J. Heering, and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Computing Surveys*, vol. 37, no. 4, pp. 316–344, 2005.

[16] T. Parr and K. Fisher, "LL(*): The foundation of the ANTLR parser generator," in* Proc. 32nd ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2011, pp. 425–436.

[17] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 1–19, 1970.

[18] G. A. Kildall, "A unified approach to global program optimization," in *Proc. 1st ACM Symp. Principles of Programming Languages (POPL)*, 1973, pp. 194–206.

[19] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[20] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.